

PV 大题--专题突破

写在前面：

PV 大题考查使用伪代码控制进程之间的同步互斥关系，它需要我们一定的代码分析能力，算法设计能力，有时候会给你一段伪代码让你补全使用信号量控制的操作，**请一定不要相信某些人告诉你只要背一个什么模板，记住什么套路就能拿下这个大题，这是不切实际的！！**

在真题的考查中，题目是较为灵活的，需要我们临场分析，如果只要背一背就能拿满分没有区分度，那出题老头也没必要年年都考 PV 了对吧？因此在正式进入大题讲解之前，必须对一些前置知识理解透彻，比如什么是同步与互斥，信号量的分类，以及考纲上的三个模型（生产者消费者、读者写者、哲学家进餐）尤其需要彻底搞懂！！

这里我会大家学习考查的最多的模型—消费者生产者模型，从真题来看，至少有三分之二以上的 PV 问题都是基于此模型进行拓展（比如加一些资源，多一些同步关系等）。并且大家在分析其他模型的时候，也要达到一个标准：

1. 能够理解清楚每一行 PV 的意义是什么，为什么这里需要同步，这里要互斥。
2. 能够熟练手写考纲上三个模型的代码，不允许出错。
3. 真题考查的问题，一定要想一下它和我们已经学过的模型的区别与联系，只有多思考才能在下一次考查到相同类型问题的时候能够熟练写出来。

前置知识

1. 临界资源：一次只允许一个进程去访问，比如打印机。
2. 临界区：进程对于临界资源访问的程序成为临界区，一般需要加上“互斥信号量”（下面会讲）。
3. 互斥关系：由于进程的并发性同时去访问同一个临界资源会出现问题，需要互斥地去访问。比如两个进程同时访问一个打印机，数据会错乱。
4. 同步关系：进程之间的操作事件存在时序关系，比如先后关系，我们在真题会遇到这种说法“操作 C 必须在操作 A、B 之后完成”，也就是在告诉你这里有一个同步关系，需要你用信号量去定义。
5. 信号量：对于信号量，我们不要去研究其复杂的概念，它的核心就是：**阻塞—唤醒**，我们在程序设计的时候也只要思考：**我这个信号量是为了阻塞什么操作，什么时候会被唤醒，因此 PV 总是成对出现，这是本质。**
 - a. 互斥信号量：初始值为 1，取值只能为 0 或 1，当有进程进入临界区时就要上锁，也就是 P 一下互斥信号量，离开的时候就 V 一下互斥信号量，千万不要忘记释放。
 - b. 同步信号量：这个名字是我自己编的，仅仅是更利于我们做题。它的初始值需要根据题目去分析，本质是用来实现进程之间的同步操作，也就是阻塞—唤醒功能，当它的值 ≤ 0 时，说明资源已经被分配完，此时进程就不能获取该资源，需要等待其他进程去唤醒。比如我们的盒子里能放三块巧克力了，杰尼龟投放巧克力，小火龙吃巧克力；当盒子里没有巧克力但是小红还想要吃，就会被阻塞住，直到杰尼龟投放了一块巧克力去唤醒小火龙告诉她：“嘿小火龙，巧克力来了，可以享用了！”。

6. 注意：初学的时候，我们可能会困惑与同步与互斥的区别，千万不要陷入进去，因为在黑书中明确定义了同步与互斥关系在一些复杂的问题中往往是交叉的，不用区分，只要能定义出信号量会做题即可。且这类题型的答案往往也不是唯一的，大家设计的算法当然有所不同，合理即给分。

解题技巧

在大题中，我们只会用到者两种信号量，不要害怕，非此即彼。**互斥信号量初始值恒为 1，而同步信号量，需要我们去设计。**

- 有一些题会初始化为**资源的个数**，比如缓冲区有 10 个空位，empty 信号量就定为 10。
- 有一些题会初始化为 0，比如我们要实现一些**"强同步"**关系的时候，像等待叫号与叫号服务。
- 有一些题我们画出一个**前驱图**即可快速解决（类似于数据结构的拓扑排序）。
- 程序中 PV 数量必须相同（**PV 成对**），请仔细检查。
- 对于单个进程，往往进需要定义一个同步互斥量，而对于两个进程需要一对，三个进行就用三个，如生产者消费者模型，等会用例题和真题带大家感受。（不想思考的时候直接用，无敌！）
- 同步信号量一定要写在互斥信号量外侧（否则会死锁）。
- 尽可能的少拿资源，比如一个题他的逻辑是水缸里有水，才能用水桶打水，那么我们在写伪代码的时候，就可以去提前判断一下水缸里是否有水（P 一下），如果没有那我们就不要去拿水桶了。**也就是说当出现多同步关系的时候，我们要去思考一下顺序。**
- **一个进程中，如果想实现“同步”关系，往往采用互斥信号量实现**，这也是我为什么在前面就说在操作系统中同步和互斥有些情况是不做区分的，**互斥就是特殊的同步**。因此一个题中互斥信号量不一定只需要一个，不要教条。

当然这些都是我的经验之谈只能作为锦上添花，具体的还要在题目中才能分享给你们。

基本设计模板：

```
// 互斥访问临界区
A () {
    P(mutex);
    操作;
    V(mutex);
}
```

```
// 先A后B同步关系
A () {
    操作;
    V(同步信号量);
}

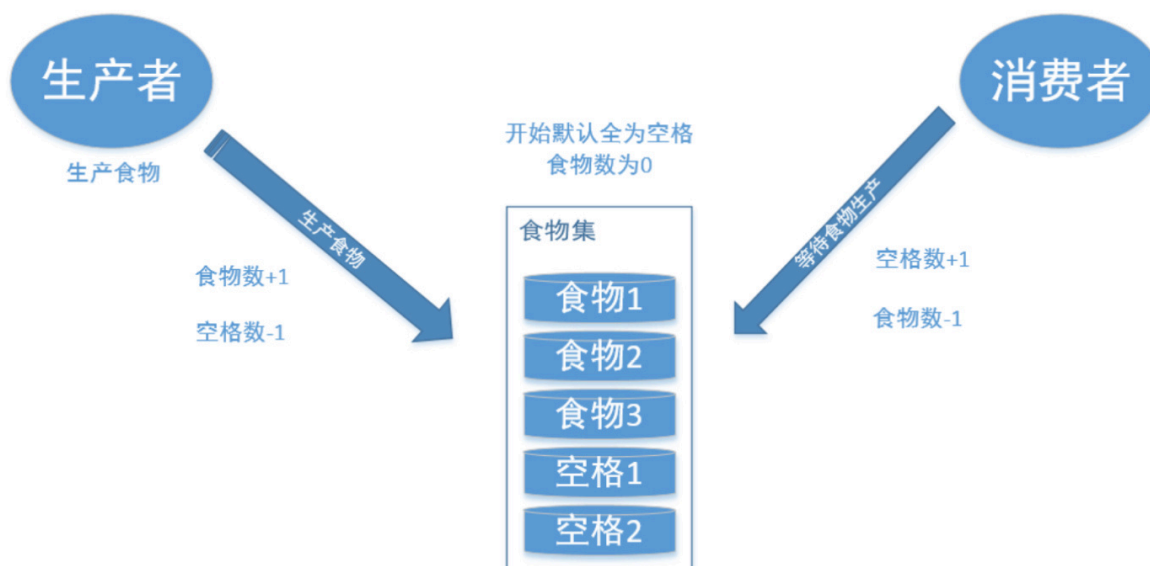
B () {
    P(同步信号量);
    操作;
}
```

生产者-消费者模型 🌟🌟

分析方法：所有的 PV 大题，都按照以下步骤进行分析，这里用生产者消费者模型来举例。

1. 找到题目中的进程个数，以及资源的个数
2. 分析各个进程之间的同步关系，找到需要互斥访问的临界区
3. 定义出互斥信号量和同步信号量
4. 根据进程同步关系，写出伪代码
5. 检查 PV 是否成对，是否有死锁问题（对 mutex 操作放在中间）
6. 模拟一下自己的伪代码，看看是否有遗漏的信号量（如果自信可不做）

生产者消费者模型



我们来看这个模型，如图所示有一个缓冲区，假设可以容纳 n 个产品，生产者负责投放产品，而消费者负载消费产品，图中的产品就是食物。请用 PV 操作，实现同步互斥关系，要求给出伪代码。

分析：

1. 两类进程：生产者、消费者
2. 两类资源：大小为 n 的缓冲区，产品（注意两类资源不一定只需要两个信号量，需要根据关系而定）
3. 关系分析：
 - a. 互斥访问缓冲区
 - b. 缓冲区有空位置才能生产，有产品才能消费
4. 信号量定义，并写出伪代码（直接看我写的）

```

semaphore mutex=1, empty=n, full=0; //初始有10个空位, 0个产品
void producer() {
    while(1) { //不断重复生产
        生产1个产品;
        p(empty); //判断是否有空位,
                没有就阻塞.
        p(mutex); 对缓冲区上锁
        放入产品;
        v(mutex); 解锁(别忘了)
        v(full); 产品+1, 去唤醒
                被阻塞的消费者
    }
}

void consumer() {
    while(1) {
        p(full); //判断是否有产品
        p(mutex);
        取产品;
        v(mutex);
        v(empty); 空位+1, 唤醒生产者
        消费产品;
    }
}

```

Q: 想一想能不能把对于缓冲区上锁的操作与 P (同步互斥量) 的操作互换?

不行, 会死锁。举个例子, 小火龙没有判断盒子里有没有巧克力, 就直接获取盒子, 再去判断有没有巧克力, 但是此时盒子里没有巧克力, 因此小火龙被阻塞住了, 也没有释放盒子; 这时杰尼龟来了, 准备投放巧克力, 想获取盒子的时候发现盒子还没有被释放, 因此也被阻塞住了, 也就不能生产巧克力去唤醒小火龙, 此时进程就死锁了。

Q: 有同学可能会问了, 判断缓冲区中有没有资源, 和减一操作可以分开吗? 是不可以的, 这是原子性操作, 不可分。请看大黑书《深入理解计算机系统》原话:

- $P(s)$: 如果 s 是非零的, 那么 P 将 s 减 1, 并且立即返回。如果 s 为零, 那么就挂起这个线程, 直到 s 变为非零, 而一个 V 操作会重启这个线程。在重启之后, P 操作将 s 减 1, 并将控制返回给调用者。
 - $V(s)$: V 操作将 s 加 1。如果有任何线程阻塞在 P 操作等待 s 变成非零, 那么 V 操作会重启这些线程中的一个, 然后该线程将 s 减 1, 完成它的 P 操作。
- P 中的测试和减 1 操作是不可分割的, 也就是说, 一旦预测信号量 s 变为非零, 就会将 s 减 1, 不能有中断。 V 中的加 1 操作也是不可分割的, 也就是加载、加 1 和存储信号量的过程中没有中断。注意, V 的定义中没有定义等待线程被重启的顺序。唯一的要求是 V 必须只能重启一个正在等待的线程。因此, 当有多个线程在等待同一个信号量时, 你不能预测 V 操作要重启哪一个线程。

至此, 我们已经搞懂了 PV 大题的本质, 学习了最常见的模型, 剩下的就是通过真题的训练去摸索出套路, 熟能生巧, 同时也要多思考模型的设计, 以不变应万变。

1.【2009 统考真题】三个进程 P_1, P_2, P_3 互斥使用一个包含 N ($N > 0$) 个单元的缓冲区。 P_1 每次用 `produce()` 生成一个正整数并用 `put()` 送入缓冲区某一空单元； P_2 每次用 `getodd()` 从该缓冲区中取出一个奇数并用 `countodd()` 统计奇数个数； P_3 每次用 `geteven()` 从该缓冲区中取出一个偶数并用 `counteven()` 统计偶数个数。请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义的信号量的含义(要求用伪代码描述)。

分析：进程： P_1, P_2, P_3 资源：缓冲区，奇数，偶数。

同步关系：有空位 能放，有奇数/偶数 能取。

定义：semaphore `mutex` = 1, `empty` = N , `odd` = 0, `even` = 0;

```

P1 {
    while(1) {
        p(empty);
        p(mutex);
        num = produce();
        put();
        if (num % 2 == 0) {
            V(even);
        }
        else V(odd);
        V(mutex);
    }
}

```

```

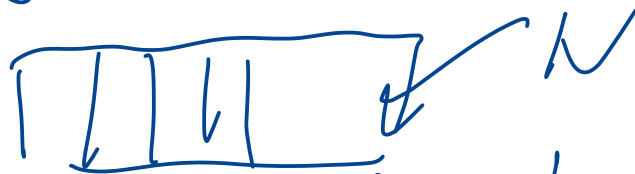
P2 {
    while(1) {
        p(odd);
        p(mutex);
        getodd();
        countodd();
        V(mutex);
        V(empty);
    }
}

```

```

P3 {
    while(1) {
        p(even);
        p(mutex);
        geteven();
        counteven();
        V(mutex);
        V(empty);
    }
}

```



$N = 1000$

4. 【2014 统考真题】系统中有多个生产者进程和多个消费者进程，共享一个能存放 1000 件产品的环形缓冲区（初始为空）。缓冲区未满时，生产者进程可以放入其生产的一件产品，否则等待；缓冲区未空时，消费者进程可从缓冲区取走一件产品，否则等待。要求一个消费者进程从缓冲区连续取出 10 件产品后，其他消费者进程才可以取产品。请使用信号 P, V (wait(), signal()) 操作实现进程间的互斥与同步，要求写出完整的过程，并说明所用信号量的含义和初值。

分析：进程：多个生产者 消费者 资源：缓冲区，产品

同步关系：↓

semaphore

$\text{mutex} = 1, \text{empty} = 1000, \text{full} = 0, \text{mutex_others} = 1;$

消费者 $j()$

while(1)

$P(\text{mutex_others});$

for(int i=0; i<10; i++)

$P(\text{full});$

$P(\text{mutex});$

取产品;

$V(\text{mutex});$

$V(\text{empty});$

$V(\text{mutex_others});$

生产者 $i()$

while(1)

生产一件产品

$P(\text{empty});$

$P(\text{mutex});$

放入产品;

$V(\text{mutex});$

$V(\text{full});$